

Desenvolvimento de um Serviço de Metadata Compatível com Cloud-Init para Incus

Gustavo Santos¹, Maycon L. M. Peixoto¹

¹Departamento de Ciência da Computação – Universidade Federal da Bahia (UFBA)

{gustavoafs,mayconleone}@ufba.br

Abstract. *Public cloud providers offer instance metadata services that allow virtual machines and containers to discover their own configuration at boot time, enabling automated provisioning through cloud-init. However, private infrastructure managers such as Incus lack an equivalent native metadata service, forcing administrators to rely on manual configuration or ad-hoc solutions. This work presents the design and implementation of a cloud-init compatible metadata service for Incus, providing REST API endpoints for metadata, user-data, vendor-data, and network configuration delivery. The service uses event-driven architecture with periodic synchronization to maintain up-to-date instance information in a local database, identifying requesting instances by their source IP address. The proposed solution is evaluated in a real Incus environment, demonstrating functional compatibility with cloud-init and the ability to serve instance configuration transparently.*

Resumo. *Provedores de nuvem pública oferecem serviços de metadata de instância que permitem que máquinas virtuais e contêineres descubram sua própria configuração durante a inicialização, viabilizando o provisionamento automatizado por meio do cloud-init. No entanto, gerenciadores de infraestrutura privada como o Incus não possuem um serviço de metadata nativo equivalente, obrigando administradores a recorrer a configurações manuais ou soluções ad-hoc. Este trabalho apresenta o projeto e a implementação de um serviço de metadata compatível com cloud-init para Incus, fornecendo endpoints REST para entrega de metadata, user-data, vendor-data e configuração de rede. O serviço utiliza uma arquitetura orientada a eventos com sincronização periódica para manter informações atualizadas das instâncias em um banco de dados local, identificando as instâncias requisitantes pelo endereço IP de origem. A solução proposta é avaliada em um ambiente real com Incus, demonstrando compatibilidade funcional com o cloud-init e a capacidade de servir configurações de instância de forma transparente.*

1. Introdução

A computação em nuvem transformou profundamente a forma como infraestruturas são provisionadas e gerenciadas. Provedores públicos como Amazon Web Services (AWS), Google Cloud Platform (GCP) e Microsoft Azure consolidaram um modelo operacional no qual máquinas virtuais e contêineres são criados, configurados e descartados de forma programática e automatizada. Central a esse modelo está o *Instance Metadata Service* (IMDS), um serviço interno acessível via um endereço IP reservado — tipicamente 169.254.169.254, pertencente à faixa de endereços *link-local* IPv4 definida pela

RFC 3927 Cheshire et al. [2005] — que permite a cada instância descobrir sua própria configuração em tempo de inicialização: nome do host, endereços IP, chaves SSH autorizadas e *scripts* de *user-data* Amazon Web Services [2024a]. Essa capacidade é o alicerce sobre o qual ferramentas de provisionamento automatizado, especialmente o *cloud-init*, operam para configurar instâncias sem intervenção manual Canonical Ltd. [2024a].

Gerenciadores de contêineres e máquinas virtuais voltados à infraestrutura privada, como o Incus — projeto derivado (*fork*) do LXD, mantido pela organização Linux Containers Linux Containers [2024]; Gräber [2023] — carecem de um serviço de metadados nativo equivalente ao oferecido pelas nuvens públicas. Essa ausência obriga administradores a configurar instâncias manualmente ou a recorrer a *scripts* ad hoc que não seguem nenhum padrão estabelecido, rompendo o fluxo de provisionamento esperado pelo *cloud-init* e inviabilizando a portabilidade de configurações entre ambientes públicos e privados. O resultado prático é que ambientes Incus, por mais capazes que sejam em termos de virtualização, tornam-se cidadãos de segunda classe no ecossistema de automação de infraestrutura.

Embora soluções como o serviço de metadados do OpenStack abordem esse problema no contexto de plataformas de nuvem privada completas OpenStack Foundation [2024], não existe uma solução leve e independente projetada especificamente para ambientes Incus. Isso é particularmente relevante para cenários de *homelab*, computação de borda (*edge computing*) e nuvens privadas de pequena escala, nos quais implantar uma plataforma completa como o OpenStack representa uma complexidade operacional desproporcional às necessidades do ambiente. Existe, portanto, uma lacuna técnica não atendida pelas soluções existentes.

Este trabalho propõe o desenvolvimento de um serviço de metadados compatível com *cloud-init* para o Incus. O serviço implementa uma API REST que entrega metadados, *user-data*, *vendor-data* e configuração de rede às instâncias, seguindo a interface esperada pelo *cloud-init* Canonical Ltd. [2024a]. A arquitetura adota um modelo orientado a eventos combinado com sincronização periódica, garantindo que os dados das instâncias permaneçam atualizados de forma consistente com o estado reportado pelo hipervisor Incus. O serviço foi concebido para ser implantado de forma autônoma, sem dependência de componentes externos de grande porte, atendendo diretamente ao perfil de ambientes de infraestrutura privada de menor escala.

As principais contribuições deste trabalho são: (a) uma análise das implementações de serviços de metadados em nuvens públicas e privadas, identificando os requisitos mínimos para compatibilidade com o *cloud-init*; (b) o projeto e a implementação de um serviço de metadados compatível com *cloud-init* para o Incus, incluindo a modelagem da API, o mecanismo de sincronização de instâncias e a persistência dos dados; e (c) a avaliação do serviço em ambientes Incus reais, verificando a correteza do provisionamento automatizado. O restante deste trabalho está organizado da seguinte forma: a Seção 2 apresenta os trabalhos relacionados; a Seção 3 detalha a proposta e a implementação; a Seção 4 apresenta a avaliação experimental; e a Seção 5 conclui o trabalho.

2. Trabalhos Relacionados

Esta seção apresenta os principais trabalhos e tecnologias relacionados ao serviço de metadados proposto neste trabalho. São discutidos o mecanismo de inicialização de instân-

cias cloud-init, os serviços de metadados das principais nuvens públicas e privadas, e as limitações atuais do Incus em relação a esse ecossistema.

2.1. Cloud-init e Inicialização de Instâncias

O cloud-init é o padrão da indústria para a inicialização automatizada de instâncias em ambientes de nuvem Canonical Ltd. [2024a]. Desenvolvido originalmente pela Canonical e amplamente adotado pelos principais provedores de infraestrutura, o cloud-init é executado durante a inicialização do sistema operacional e é responsável por configurar aspectos fundamentais da instância, tais como nome de host, usuários e grupos, chaves SSH, pacotes de software, scripts de inicialização e parâmetros de rede.

O processo de inicialização do cloud-init é organizado em quatro estágios sequenciais, cada um com responsabilidades bem definidas Canonical Ltd. [2024a]: (1) `init-local`, executado antes da configuração de rede, responsável por identificar o *datasource* a partir de fontes locais como disco, DMI e linha de comando do *kernel*; (2) `init (rede)`, executado após a rede estar ativa, responsável por buscar os dados de configuração do *datasource* identificado; (3) `modules-config`, que aplica os módulos de configuração como criação de usuários e instalação de pacotes; e (4) `modules-final`, que executa *scripts* finais e mensagens de conclusão. Essa sequência é relevante porque impõe requisitos temporais à disponibilidade do serviço de metadados: os dados devem estar acessíveis antes do estágio `init`, momento em que o cloud-init realiza a busca efetiva dos metadados.

O funcionamento do cloud-init é baseado no conceito de *datasources*: fontes de dados a partir das quais a ferramenta obtém as informações necessárias para configurar a instância. Cada *datasource* é composto por quatro tipos de dados distintos: (i) *meta-data*, que contém informações de identidade da instância como identificador único, nome de host e dados de rede; (ii) *user-data*, que permite ao usuário final fornecer configurações personalizadas, como *scripts* shell ou arquivos de configuração no formato cloud-config; (iii) *vendor-data*, fornecido pelo provedor de infraestrutura para configurações específicas da plataforma; e (iv) *network-config*, que define a configuração de rede da instância em formatos padronizados Canonical Ltd. [2024a]. Dentre esses dados, o campo `instance-id` é particularmente crítico: o cloud-init utiliza-o para determinar se a inicialização corrente é a primeira (*first boot*), acionando a configuração completa, ou uma inicialização subsequente, na qual módulos já aplicados são ignorados.

A capacidade do cloud-init de abstrair as particularidades de cada plataforma por meio da camada de *datasources* é o que o torna uma solução portátil e amplamente adotada. No entanto, para que essa abstração funcione corretamente, é necessário que a infraestrutura subjacente disponibilize um endpoint de metadados que o cloud-init possa consultar durante a inicialização.

2.2. Datasource NoCloud

O *datasource NoCloud* é o mais flexível dentre os oferecidos pelo cloud-init e é diretamente relevante para ambientes de infraestrutura privada Canonical Ltd. [2024f]. Ele permite obter dados de configuração a partir de uma URL HTTP base (*seedfrom*), consultando os caminhos `/meta-data`, `/user-data`, `/vendor-data` e `/network-config` relativos a essa URL. Essa abordagem não exige que o serviço implemente a hierarquia

de caminhos versionados do EC2 (como `/latest/meta-data/`), bastando que os quatro endpoints estejam disponíveis na raiz configurada. O *NoCloud* suporta os protocolos HTTP, HTTPS, FTP e FTPS, e requer apenas que o arquivo meta-data contenha, no mínimo, os campos `instance-id` e `local-hostname` Canonical Ltd. [2024f]. Essa simplicidade torna o *NoCloud* especialmente adequado para serviços de metadados independentes de plataforma, como o proposto neste trabalho.

2.3. AWS Instance Metadata Service (IMDS)

A Amazon Web Services (AWS) disponibiliza o *Instance Metadata Service* (IMDS), acessível pelas instâncias EC2 por meio do endereço IP de link-local `169.254.169.254` Amazon Web Services [2024a]. Esse serviço expõe uma API HTTP que permite às instâncias consultar informações sobre si mesmas sem necessidade de credenciais adicionais.

O IMDS oferece duas versões com características distintas de segurança. A primeira versão, IMDSv1, opera com requisições GET simples sem autenticação, o que pode expor a instância a ataques de falsificação de requisição do lado do servidor (*Server-Side Request Forgery* — SSRF) MITRE Corporation [2021]. A segunda versão, IMDSv2, introduzida em 2019, adota um modelo baseado em sessões: a instância deve primeiro obter um token de acesso por meio de uma requisição PUT com cabeçalho `X-aws-ec2-metadata-token-ttl-seconds`, e em seguida utilizar esse token nas requisições subsequentes via o cabeçalho `X-aws-ec2-metadata-token` Amazon Web Services [2024b]. Essa abordagem mitiga significativamente os riscos associados a ataques SSRF, uma vez que a maioria das vulnerabilidades SSRF permite apenas requisições GET e não possibilita a inclusão de cabeçalhos personalizados. Além disso, o IMDSv2 aplica um limite de saltos (*hop limit*) de 1 nas respostas PUT e rejeita requisições contendo o cabeçalho `X-Forwarded-For`, bloqueando ataques baseados em proxies reversos Amazon Web Services [2019].

Os dados servidos pelo IMDS incluem, entre outros: identificador único da instância (`instance-id`), nome de host, endereços MAC das interfaces de rede, metadados de interfaces de rede, chaves SSH públicas associadas à instância e o *user-data* fornecido no momento do lançamento. A API é organizada em um namespace hierárquico acessível via caminhos como `/latest/meta-data/` e `/latest/user-data`.

2.4. GCP Metadata Server

O Google Cloud Platform (GCP) disponibiliza o *Metadata Server*, acessível pelo nome de host `metadata.google.internal` (resolvido para `169.254.169.254`) e também diretamente pelo endereço IP `169.254.169.254` Google Cloud [2024]. O serviço é organizado em dois escopos principais: metadados de projeto, que contêm informações compartilhadas entre todas as instâncias de um projeto, e metadados de instância, específicos de cada máquina virtual.

Um aspecto de segurança relevante do Metadata Server do GCP é a exigência do cabeçalho HTTP `Metadata-Flavor: Google` em todas as requisições. Requisições que não incluam esse cabeçalho são rejeitadas pelo serviço, o que serve como mecanismo de proteção contra ataques SSRF oriundos de contextos externos à instância Google Cloud [2024]. Mecanismo semelhante é adotado pelo Azure Instance Metadata Service, que exige o cabeçalho `Metadata: true` Microsoft [2024]. Embora esses cabeçalhos

obrigatórios mitiguem ataques SSRF simples — que tipicamente não permitem a inclusão de cabeçalhos personalizados — oferecem proteção inferior ao modelo de sessões do IMDSv2 da AWS Amazon Web Services [2019]. Além da proteção contra SSRF, o serviço do GCP suporta notificações de alterações de metadados via mecanismo de *long polling*, permitindo que aplicações reajam dinamicamente a mudanças de configuração.

2.5. OpenStack Metadata Service

O OpenStack, principal plataforma de nuvem privada de código aberto, implementa um serviço de metadados compatível com a API EC2 da AWS, também disponível no endereço 169.254.169.254 OpenStack Foundation [2024]. O serviço oferece dois endpoints distintos: o endpoint compatível com EC2 (`/latest/meta-data/`) e o endpoint nativo do OpenStack (`/openstack/`), que serve dados em formato JSON estruturado conforme o modelo de dados da plataforma.

Uma característica fundamental do serviço de metadados do OpenStack é seu forte acoplamento com os demais componentes da plataforma. O roteamento de requisições de metadados é realizado pelo Neutron, o componente de rede do OpenStack, que intercepta requisições destinadas ao endereço 169.254.169.254 e as encaminha ao serviço correto com base na interface de rede da instância solicitante. O provisionamento dos dados de metadados, por sua vez, é gerenciado pelo Nova, o componente de computação. Esse acoplamento implica que o serviço de metadados do OpenStack não pode ser executado de forma autônoma fora do ecossistema OpenStack OpenStack Foundation [2024], o que limita sua aplicabilidade em outros contextos de virtualização. Em contextos acadêmicos, a dependência do ecossistema completo é aceita quando a escala justifica: Bollig et al. [2018] descrevem a plataforma Stratus, uma nuvem privada baseada em OpenStack na Universidade de Minnesota, na qual o cloud-init consulta o serviço de metadados do Nova durante a inicialização para configurar endereços IP, chaves SSH e, por meio do *vendor-data*, aplicar atualizações de segurança obrigatórias em todas as máquinas virtuais. Esse exemplo ilustra como o padrão de metadata service é fundamental para operações de nuvem privada, mas também evidencia que sua adoção requer a implantação de uma plataforma completa como o OpenStack.

2.6. Incus/LXD e a Lacuna de Metadados

O Incus é um gerenciador moderno de contêineres de sistema e máquinas virtuais, surgido em 2023 como um *fork* da comunidade do projeto LXD, originalmente desenvolvido e mantido pela Canonical Gräber [2023]; Linux Containers [2024]. O projeto foi iniciado por Stéphane Gräber, criador original do LXD, após a Canonical encerrar a participação comunitária no desenvolvimento do projeto Gräber [2023]. O Incus herda a arquitetura e as funcionalidades do LXD, com o objetivo de manter o desenvolvimento aberto e orientado pela comunidade Canonical Ltd. [2024d].

O Incus possui suporte nativo para configurações do cloud-init por meio de chaves de configuração específicas: `cloud-init.user-data` para dados de usuário, `cloud-init.vendor-data` para dados do fornecedor e `cloud-init.network-config` para configuração de rede Linux Containers [2024]. Essas configurações podem ser definidas diretamente na instância ou propagadas via perfis de configuração. No entanto, o Incus não disponibiliza nenhum endpoint HTTP de metadados que as instâncias possam consultar durante a inicialização.

A entrega das configurações do cloud-init no Incus ocorre por meio do *datasource LXD*, que se comunica com o hipervisor por um *socket* Unix em `/dev/incus/sock` (ou `/dev/lxd/sock` no LXD original) Canonical Ltd. [2024c]. Esse *socket* é provido pelo *Incus agent*, um processo leve executado dentro das instâncias. Alternativamente, as configurações podem ser injetadas diretamente no sistema de arquivos da instância via *config drive* Linux Containers [2024]. Embora esses mecanismos sejam funcionais para casos de uso básicos, eles apresentam uma limitação temporal crítica: o *Incus agent* é iniciado tardiamente na sequência de *boot* da máquina virtual, de modo que os estágios iniciais do cloud-init (*ds-identify* e *init-local*) frequentemente são executados antes que o *socket* esteja disponível. Essa condição de corrida resulta em falhas na detecção do *datasource* e, consequentemente, na configuração automática da instância.

Trabalhos como o de Sousa Sousa [2018], que implementa uma infraestrutura de nuvem privada na USP utilizando LXD com OpenNebula para instanciação de contêineres sob demanda, evidenciam que a plataforma é viável para uso acadêmico e de pesquisa, mas a inicialização das instâncias depende de *scripts* definidos manualmente nos *templates* da plataforma de orquestração, sem um serviço de metadados padronizado.

Além dessa limitação temporal, os mecanismos atuais do Incus não replicam o fluxo completo do *datasource* do cloud-init que opera via requisições HTTP ao endereço 169.254.169.254. Isso impede o uso do *datasource NoCloud* HTTP Canonical Ltd. [2024f] ou do *datasource EC2* Canonical Ltd. [2024b], que são os mais amplamente suportados e testados no ecossistema cloud-init, e limita a compatibilidade com imagens e ferramentas de automação que esperam um serviço de metadados acessível via rede.

2.7. Algoritmo de Consenso RAFT

O RAFT é um algoritmo de consenso distribuído projetado com ênfase na compreensibilidade, proposto por Ongaro e Ousterhout Ongaro and Ousterhout [2014]. Ao contrário do Paxos, que é reconhecidamente difícil de compreender e implementar corretamente, o RAFT decompõe o problema do consenso em subproblemas relativamente independentes: eleição de líder, replicação de log e segurança. Essa decomposição facilita tanto o entendimento do algoritmo quanto a construção de implementações corretas.

No RAFT, um cluster é composto por um nó líder e um ou mais seguidores. O líder é responsável por receber requisições de escrita, replicá-las para os seguidores via entradas de log, e confirmar a operação assim que uma maioria de nós tiver persistido a entrada. Essa garantia de quórum assegura que o sistema tolera falhas de até $\lfloor (n - 1)/2 \rfloor$ nós em um cluster de n membros, mantendo disponibilidade e consistência.

A biblioteca `hashicorp/raft` HashiCorp [2024] é uma implementação em Go amplamente utilizada do protocolo RAFT, desenvolvida e mantida pela HashiCorp. Ela fornece uma API estável para integração do consenso distribuído em aplicações, sendo empregada em projetos de destaque como o Consul e o Vault. Neste trabalho, a biblioteca é utilizada para prover alta disponibilidade ao serviço de metadados, garantindo que o estado do cluster permaneça consistente mesmo diante de falhas de nós individuais.

2.8. Segurança em Serviços de Metadados

Serviços de metadados representam uma superfície de ataque relevante em ambientes de nuvem, uma vez que expõem informações sensíveis — como credenciais temporárias,

chaves SSH e configurações de rede — por meio de um endpoint HTTP acessível sem autenticação convencional. O vetor de ataque mais significativo é o *Server-Side Request Forgery* (SSRF): caso uma aplicação web executada dentro da instância possua uma vulnerabilidade que permita a um atacante induzir requisições HTTP arbitrárias, o atacante pode direcionar essas requisições ao endereço 169.254.169.254 e obter os metadados da instância MITRE Corporation [2021]. Esse vetor é catalogado pela MITRE ATT&CK como a técnica T1552.005 (*Unsecured Credentials: Cloud Instance Metadata API*).

Um caso emblemático que ilustra a gravidade dessa classe de ataque é a violação de dados da Capital One, ocorrida em março de 2019 e analisada em profundidade por Novaes Neto et al. Novaes Neto et al. [2020]. Nesse incidente, um atacante explorou uma vulnerabilidade SSRF em um *Web Application Firewall* (WAF) mal configurado para induzir requisições ao IMDSv1 da AWS no endereço 169.254.169.254, obtendo credenciais temporárias IAM associadas à instância. Com essas credenciais, o atacante acessou buckets S3 contendo dados de 106 milhões de clientes, resultando em prejuízos superiores a 150 milhões de dólares. A cadeia de ataque (SSRF, acesso ao IMDS sem autenticação, exfiltração de credenciais) demonstrou que a simplicidade do IMDSv1, baseado em requisições GET sem qualquer verificação, representava um risco sistêmico. Esse incidente foi um dos catalisadores para a introdução do IMDSv2 pela AWS.

Os provedores de nuvem adotam diferentes estratégias de mitigação. A AWS introduziu o IMDSv2 com autenticação baseada em sessões Amazon Web Services [2019], o GCP exige o cabeçalho `Metadata-Flavor: Google` Google Cloud [2024], e o Azure requer o cabeçalho `Metadata: true` Microsoft [2024]. Em ambientes de infraestrutura privada, como os atendidos pelo serviço proposto neste trabalho, o risco de SSRF é tipicamente menor, pois o serviço de metadados não está exposto à internet e o perímetro de rede é controlado pelo administrador. Ainda assim, a documentação do risco e a adoção de boas práticas — como a restrição de acesso por endereço IP de origem e a possibilidade futura de autenticação por token — são medidas relevantes para ambientes de produção.

2.9. Comparação entre as Abordagens

A Tabela 1 sintetiza as principais características dos serviços de metadados discutidos nesta seção em comparação com a proposta deste trabalho. Os critérios avaliados são: operação autônoma (sem dependência de plataforma), compatibilidade com o cloud-init via endpoint HTTP, suporte a contêineres de sistema, suporte a máquinas virtuais, alta disponibilidade e disponibilidade como software de código aberto.

2.10. Considerações Finais

A análise dos trabalhos relacionados evidencia uma lacuna tecnológica relevante: não existe atualmente um serviço de metadados leve e autônomo para o Incus que implemente o protocolo HTTP de metadados esperado pelo cloud-init, habilitando o fluxo completo de datasource para contêineres e máquinas virtuais gerenciados por essa plataforma. Os serviços existentes nas nuvens públicas (AWS IMDS e GCP Metadata Server) são proprietários e estão acoplados às respectivas infraestruturas. O serviço de metadados do OpenStack, apesar de ser código aberto e compatível com cloud-init, depende integralmente do ecossistema OpenStack para funcionar, inviabilizando seu uso isolado. O próprio Incus, por sua vez, oferece mecanismos de entrega de configuração baseados no datasource LXD Canonical Ltd. [2024c] que, além de não cobrirem o fluxo completo de

Tabela 1. Comparação entre serviços de metadados para cloud-init.

Solução	Autônomo	Compat. cloud-init	Suporta Contêineres	Suporta VMs	Alta Disp.	Código Aberto
AWS IMDS	✗	✓	✗	✓	✓	✗
GCP Metadata	✗	✓	✗	✓	✓	✗
OpenStack Metadata	✗	✓	✗	✓	✓	✓
LXD/Incus (atual)	✓	✗	✓	✓	✗	✓
Este trabalho	✓	✓	✓	✓	✓	✓

consulta HTTP, sofrem de uma condição de corrida temporal durante a inicialização. A necessidade de ferramentas para inicialização automatizada de instâncias em nuvens de infraestrutura é documentada na literatura desde 2011, quando Bresnahan et al. [2011] propuseram o cloudinit.d, uma ferramenta para lançamento coordenado de conjuntos de máquinas virtuais interdependentes em nuvens IaaS, evidenciando já naquele período a dificuldade de configurar instâncias de forma repetível e automatizada sem o suporte de serviços de metadados. A adoção crescente de práticas de Infraestrutura como Código (*Infrastructure as Code*, IaC) Guerriero et al. [2019]; Artač et al. [2017] reforça a importância de serviços de metadados padronizados para o provisionamento automatizado de instâncias. Nesse contexto, Teppan et al. [2022] observam que, dentre as categorias de ferramentas *cloud-native* catalogadas pela CNCF, a categoria de provisionamento é a que menos dispõe de projetos maduros para ambientes *on-premise*, o que corrobora a lacuna identificada neste trabalho para o ecossistema Incus.

Este trabalho propõe um serviço de metadados compatível com cloud-init, projetado especificamente para ambientes Incus, que opera de forma autônoma, suporta tanto contêineres quanto máquinas virtuais, oferece alta disponibilidade via o algoritmo de consenso RAFT Ongaro and Ousterhout [2014] implementado pela biblioteca hashicorp/raft HashiCorp [2024], e é disponibilizado como software de código aberto. Dessa forma, preenche a lacuna identificada e viabiliza o uso do ecossistema completo do cloud-init em infraestruturas baseadas em Incus.

3. Proposta

Este trabalho propõe o desenvolvimento de um serviço de metadados compatível com cloud-init voltado a instâncias gerenciadas pelo Incus Linux Containers [2024]. O objetivo é viabilizar a configuração automática de contêineres e máquinas virtuais no momento de sua inicialização, por meio do protocolo de metadados já consolidado pelo cloud-init Canonical Ltd. [2024a], sem a necessidade de configuração manual por instância.

O cloud-init é amplamente utilizado em ambientes de nuvem pública para realizar tarefas como definição de hostname, injeção de chaves SSH, configuração de rede e execução de scripts na primeira inicialização. Contudo, o Incus não oferece nativamente um serviço de metadados acessível via HTTP pelas instâncias, o que impede o uso do

cloud-init em seu modo de operação padrão baseado em datasource de rede. A proposta preenche essa lacuna ao expor uma API REST que as instâncias podem consultar durante o boot, identificando-se pelo próprio endereço IP de origem.

3.1. Arquitetura

O serviço segue uma arquitetura modular, com responsabilidades bem delimitadas entre seus componentes. A Figura 1 ilustra a interação entre os principais módulos do sistema.

Os componentes são organizados da seguinte forma:

- **cmd/server**: Ponto de entrada da aplicação. Responsável por inicializar e conectar todos os demais componentes, incluindo o banco de dados, o barramento de eventos, o agendador de tarefas e o servidor HTTP.
- **internal/api**: Camada HTTP da aplicação, implementada com o framework Gin Gin-Gonic Contributors [2024]. Divide-se em dois grupos de rotas: endpoints públicos, acessíveis pelas instâncias em `/configs/*`, que entregam os dados de metadados, user-data, vendor-data e network-config; e endpoints internos em `/internal/*`, destinados a operações de gerenciamento.
- **internal/events**: Módulo de sincronização orientada a eventos, utilizando o GoEventBus. Implementa um modelo de publicação e assinatura (*pub/sub*) que desacopla o agendamento da execução da sincronização com o Incus.
- **internal/storage**: Camada de persistência baseada em SQLite Hipp [2024], com queries geradas de forma estática e com segurança de tipos pelo SQLC. Armazena os registros de instâncias, seus metadados cloud-init, user-data, vendor-data e configurações de rede.
- **pkg/types**: Definições de tipos de dados utilizados em toda a aplicação, garantindo compatibilidade com as estruturas esperadas pelo cloud-init.

3.2. Sincronização de Instâncias

Para que o serviço consiga responder corretamente às requisições das instâncias, é necessário manter um estado atualizado de cada instância gerenciada pelo Incus. Isso é realizado por meio de um ciclo de sincronização periódico e orientado a eventos, descrito a seguir:

1. Um agendador de tarefas (*cron*) é executado a cada 10 segundos e publica um evento `instances.sync` no barramento de eventos.
2. O handler do evento `instances.sync` consulta a API do Incus e obtém a lista completa de instâncias ativas no momento.
3. Para cada instância retornada, um evento individual `instance.sync` é publicado, permitindo que o processamento ocorra de forma desacoplada e paralela.
4. O handler do evento `instance.sync` executa as seguintes etapas para cada instância:
 - Extrai informações de rede do estado da instância reportado pelo Incus, incluindo endereços IPv4, IPv6 e endereço MAC de cada interface;
 - Cria ou atualiza o registro da instância no banco de dados;
 - Constrói e persiste os metadados cloud-init, incluindo `instance-id`, `host-name`, endereços IP, chaves SSH públicas, informações de posicionamento (*placement*) e interfaces de rede. As chaves SSH são extraídas das chaves de configuração `user.meta-data` (campo `public-keys` em YAML) e `user.ssh-keys` (lista separada por quebras de linha);

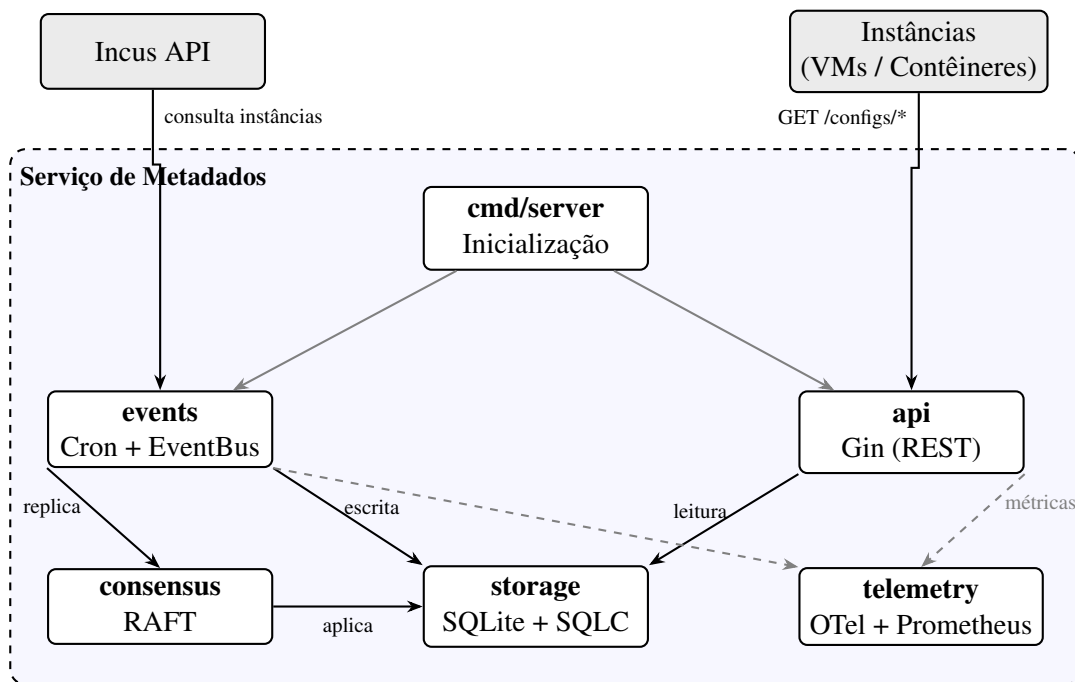


Figura 1. Arquitetura do serviço de metadados e interação entre seus componentes. Setas contínuas indicam fluxos de dados; setas tracejadas indicam instrumentação de métricas.

- Armazena o user-data a partir da chave de configuração `cloud-init.user-data` presente na configuração da instância no Incus;
- Armazena o vendor-data a partir da chave de configuração `cloud-init.vendor-data` do Incus, permitindo que perfis e configurações específicas do provedor sejam entregues por instância;
- Constrói a configuração de rede: caso a chave `cloud-init.network-config` esteja definida na configuração da instância no Incus, utiliza-a diretamente, permitindo ao administrador especificar configurações de rede estáticas, *bonds*, VLANs ou DNS personalizado; caso contrário, gera automaticamente a configuração no formato *Networking Config Version 2* Canonical Ltd. [2024e], baseada na especificação Netplan, a partir das interfaces reais reportadas pelo estado da instância;
- Atualiza o estado da instância no banco de dados, incluindo status textual e código de status numérico.

Esse modelo orientado a eventos garante que o estado armazenado permaneça consistente com o estado real das instâncias no Incus, sem impor acoplamento direto entre o agendamento e a lógica de sincronização.

3.3. Endpoints da API

Os endpoints públicos do serviço seguem o modelo do datasource *NoCloud* HTTP do cloud-init Canonical Ltd. [2024f], permitindo que instâncias consultem seus próprios dados de configuração a partir do endereço IP de origem da requisição. Os endpoints disponíveis são:

- **GET /configs/meta-data:** Retorna os metadados da instância no formato JSON ou YAML, incluindo `instance-id`, `hostname`, endereços IP, chaves SSH públicas (`public-keys`) e informações de rede. A instância é identificada pelo seu endereço IP de origem.
- **GET /configs/meta-data/:key:** Retorna um campo específico dos metadados da instância, identificado pelo parâmetro `key`.
- **GET /configs/user-data:** Retorna o `user-data` associado à instância, conforme configurado na chave `cloud-init.user-data` do Incus. O `user-data` tipicamente contém scripts ou configurações no formato `cloud-config`.
- **GET /configs/vendor-data:** Retorna dados de configuração específicos do fornecedor (*vendor-data*), que complementam o `user-data` sem sobrescrevê-lo. O serviço prioriza o `vendor-data` por instância, extraído da chave `cloud-init.vendor-data` do Incus; caso não exista, retorna o `vendor-data` global configurado pela API de gerenciamento.
- **GET /configs/network-config:** Retorna a configuração de rede da instância. Quando a chave `cloud-init.network-config` está definida na configuração da instância no Incus, o conteúdo definido pelo administrador é servido diretamente; caso contrário, uma configuração no formato *Networking Config Version 2 Canonical Ltd.* [2024e] é gerada automaticamente a partir das interfaces reais reportadas pelo Incus.

Além dos endpoints públicos, o serviço disponibiliza endpoints internos no prefixo `/internal/` para operações de gerenciamento de `vendor-data`, incluindo criação, leitura, atualização e remoção (CRUD) de registros por instância. Esses endpoints são destinados ao uso administrativo e não são expostos diretamente às instâncias.

3.4. Descoberta do Serviço pelas Instâncias

Para que o `cloud-init` nas instâncias localize o serviço de metadados de forma transparente, o serviço adota a mesma convenção utilizada pelos provedores de nuvem pública: o endereço *link-local* `169.254.169.254` Amazon Web Services [2024a]; Google Cloud [2024]. Esse endereço é adicionado à interface de ponte gerenciada pelo Incus (`incusbr0`), tornando o serviço acessível a todas as instâncias conectadas à rede do hipervisor.

Como o `cloud-init` realiza requisições na porta 80 por padrão, uma regra de redirecionamento NAT encaminha o tráfego destinado a `169.254.169.254:80` para a porta em que o serviço está em execução. Dessa forma, nenhuma configuração de rede adicional é necessária dentro das instâncias.

O datasource utilizado é o *NoCloud* em modo de rede (`dsmode: net`), configurado no perfil do Incus com o parâmetro `seedfrom` apontando para o prefixo `/configs/` do serviço. Durante a inicialização, o `cloud-init` concatena o `seedfrom` com os caminhos padrão (`meta-data`, `user-data`, `network-config`) para compor as URLs de consulta.

3.5. Tecnologias Utilizadas

A escolha das tecnologias que compõem o serviço foi guiada por critérios de desempenho, simplicidade operacional e adequação ao problema:

- **Go:** A linguagem oferece desempenho próximo ao de linguagens compiladas de baixo nível, com um modelo de concorrência baseado em goroutines que facilita

o tratamento paralelo de eventos. Além disso, o processo de compilação gera um único binário estático, simplificando o processo de implantação em ambientes de borda.

- **Gin** Gin-Gonic Contributors [2024]: Framework HTTP de alta performance para Go, com roteamento eficiente e baixo overhead. Sua API expressiva permite definir grupos de rotas e middlewares de forma clara, facilitando a separação entre os endpoints públicos e internos.
- **SQLite + SQLC** Hipp [2024]: O SQLite é um banco de dados relacional embutido, sem necessidade de processo servidor separado, adequado para serviços com requisitos de implantação simplificados. O SQLC complementa essa escolha ao gerar código Go com tipagem estática a partir de queries SQL declaradas, eliminando o uso de ORMs genéricos e reduzindo a superfície de erros em tempo de execução.
- **GoEventBus**: Biblioteca de barramento de eventos para Go que implementa o padrão publicação/assinatura. Permite desacoplar o agendamento da sincronização da lógica de processamento por instância, tornando o sistema mais modular e testável.
- **gocron**: Biblioteca de agendamento de tarefas periódicas para Go, utilizada para disparar o ciclo de sincronização a cada 10 segundos de forma confiável e configurável.
- **hashicorp/raft**: Biblioteca de consenso distribuído para Go que implementa o protocolo RAFT Ongaro and Ousterhout [2014]. Utilizada para coordenar escritas em implantações com múltiplos nós, garantindo que apenas o líder eleito realize sincronizações com o Incus e proponha alterações ao cluster.
- **OpenTelemetry + Prometheus** Cloud Native Computing Foundation [2024a,b]: O serviço utiliza o OpenTelemetry como framework de instrumentação para coleta de métricas, seguindo o padrão aberto da Cloud Native Computing Foundation (CNCF). As métricas são exportadas no formato Prometheus por meio de um endpoint `/metrics`, permitindo o monitoramento do serviço por ferramentas de observabilidade padrão do ecossistema nativo de nuvem. As métricas instrumentadas incluem a duração do processamento de eventos de sincronização (histograma `event_processing_duration_seconds`) e a contagem total de eventos processados (`event_processing_total`), ambas segmentadas por tipo de projeção e status de execução.

3.6. Consenso Distribuído via RAFT

A arquitetura de nó único apresenta um ponto único de falha (*Single Point of Failure*, SPOF): caso o nó que executa o serviço de metadados fique indisponível, todas as instâncias perdem acesso às suas configurações de inicialização, o que inviabiliza reinicializações e provisionamentos durante o período de falha. Para possibilitar implantações de alta disponibilidade em ambientes de produção, o serviço implementa consenso distribuído por meio do protocolo RAFT Ongaro and Ousterhout [2014], utilizando a biblioteca `hashicorp/raft`.

O suporte a RAFT é ativado de forma opcional por meio de variáveis de ambiente, sendo `RAFT_ENABLED=true` o ponto de entrada. Quando habilitado, o nó é configurado com os parâmetros definidos nas variáveis `NODE_ID`, `BIND_ADDR`, `DATA_DIR`, `PEERS` e `BOOTSTRAP`, permitindo compor um cluster sem alterações no binário.

O funcionamento do módulo de consenso é organizado em três camadas principais:

- **Eleição de líder:** O protocolo RAFT garante que, a qualquer momento, no máximo um nó detenha o papel de líder. Apenas o nó líder executa o ciclo de sincronização com o Incus e propõe comandos de escrita ao cluster. Os nós seguidores não realizam sincronizações independentes; caso o líder falhe, uma nova eleição é conduzida automaticamente e o nó eleito passa a assumir as responsabilidades de sincronização.
- **Replicação de log e FSM:** Cada operação de escrita (criação de instância, atualização de metadados, persistência de user-data, network-config e estado) é codificada como um comando tipado (Command) e submetida ao log replicado via `hashicorp/raft`. Após o *commit* do log em uma maioria de nós, a *Finite State Machine* (FSM) aplica o comando ao banco de dados SQLite local de cada nó, mantendo o estado consistente em todo o cluster. Os tipos de comando suportados são: `CmdCreateInstance`, `CmdUpdateInstance`, `CmdCreateOrUpdateMetadata`, `CmdCreateOrUpdateUserData`, `CmdCreateOrUpdateVendorData`, `CmdCreateOrUpdateNetworkConfig` e `CmdCreateOrUpdateState`.
- **Armazenamento persistente:** O log de entradas RAFT e o estado estável do nó são persistidos em um arquivo BoltDB localizado no diretório configurado por `DATA_DIR`. Essa escolha elimina a necessidade de um processo de banco de dados externo para o próprio mecanismo de consenso, mantendo o requisito de implantação simplificada.

Todos os nós do cluster, tanto o líder quanto os seguidores, podem responder a requisições de leitura, uma vez que cada um mantém uma cópia local e atualizada do estado no SQLite. Isso permite distribuir a carga de consultas de metadados entre os nós sem comprometer a consistência das leituras para as instâncias.

4. Resultados

Esta seção apresenta a avaliação do serviço de metadados proposto. A avaliação é organizada em cinco partes: a metodologia experimental adotada, os testes funcionais que verificam a corretude do provisionamento, os testes de desempenho que quantificam a latência e a escalabilidade do serviço, a avaliação de alta disponibilidade baseada em consenso RAFT, e uma comparação de funcionalidades com serviços de metadados de provedores de nuvem públicos.

De modo geral, os experimentos confirmam a compatibilidade funcional do serviço com o *cloud-init*: instâncias reais selecionam o *datasource* de rede, obtêm seus metadados e concluem a inicialização com sucesso. As latências de resposta situam-se na faixa de dezenas de milissegundos e, ao contrário da hipótese inicial de degradação por serialização de escrita do SQLite, o serviço manteve latência estável e ausência de erros até 200 contêineres simultâneos. A avaliação de alta disponibilidade demonstrou formação de *cluster*, replicação de estado e reeleição de líder da ordem de segundos após falha. Os detalhes de cada dimensão são apresentados a seguir.

4.1. Metodologia de Avaliação

Os experimentos foram conduzidos em uma instância de nuvem Google Compute Engine do tipo e2-standard-4, com 4 vCPUs (processador Intel Xeon a 2,20 GHz), 15 GiB de memória RAM, executando Ubuntu 24.04 LTS (*kernel* 6.17.0-1020-gcp) e Incus 6.0.0. O serviço de metadados foi implantado no mesmo host das instâncias.

Um aspecto prático relevante emergiu durante a configuração do ambiente: em provedores de nuvem pública (Google Cloud, AWS e Azure), o endereço 169.254.169.254 já é utilizado pelo próprio serviço de metadados do provedor, de modo que sua reutilização no host interfere na resolução de nomes e no acesso aos metadados da própria máquina virtual. Por essa razão, no ambiente de testes o *cloud-init* foi apontado, via *datasource* NoCloud, para o endereço de *gateway* da *bridge* do Incus (10.10.10.1:8080), diretamente alcançável pelas instâncias. Esse achado tem relevância prática por si só: a convenção do endereço reservado, embora adequada para instâncias hospedadas dentro de um provedor, não é diretamente transponível para um host que já opera sob tal provedor.

Para avaliar o comportamento sob diferentes cargas, foram criados grupos de instâncias com cardinalidades crescentes, de 5 até 200 contêineres simultâneos. Cada instância foi configurada com o *cloud-init* habilitado, apontando para o serviço de metadados proposto como fonte de configuração. Durante a inicialização, o *cloud-init* realizou requisições aos *endpoints* do serviço (*/meta-data*, */user-data*, */vendor-data* e */network-config*), seguindo o fluxo de *datasource* documentado em Canonical Ltd. [2024a].

As medições foram realizadas em três dimensões: (a) **latência de resposta da API**, correspondente ao tempo decorrido entre o envio da requisição HTTP pela instância e o recebimento da resposta; (b) **acurácia de sincronização**, verificando se os metadados armazenados refletem o estado atual das instâncias conforme reportado pelo Incus; e (c) **corretude da configuração**, avaliando se as configurações entregues foram efetivamente aplicadas pelas instâncias ao término da inicialização.

A coleta de latência foi realizada com a ferramenta *hey*, disparando 5000 requisições por configuração com concorrência de 50 requisições simultâneas a partir de um contêiner gerador de carga. Os resultados de latência são reportados nos percentis p50, p95 e p99, seguindo a prática adotada em sistemas de produção para caracterizar tanto o comportamento típico quanto os casos de cauda da distribuição.

4.2. Avaliação Funcional

A avaliação funcional verificou se o serviço de metadados entrega as informações corretas a cada instância e se essas informações são interpretadas e aplicadas com êxito pelo *cloud-init*. As propriedades testadas e seus resultados são apresentados na Tabela 2.

1. **Isolamento de metadados:** cada instância deve receber exclusivamente os metadados associados ao seu próprio identificador. Múltiplas instâncias realizaram requisições simultâneas ao serviço, e confirmou-se que o campo *local-hostname* correspondia unicamente à instância solicitante.
2. **Isolamento sob falsificação de origem:** como o serviço identifica a instância pelo endereço IP de origem da requisição, foi testada a resistência à falsificação

por meio do cabeçalho X-Forwarded-For. Confirmou-se que uma requisição forjada com o IP de outra instância não retornou os dados da vítima, validando a desativação da confiança em *proxies* (`SetTrustedProxies(nil)`).

3. **Execução de *user-data*:** um *script* de *user-data* injetado via chave `cloud-init.user-data` do Incus foi executado durante a inicialização, criando um arquivo sentinela verificado após o término do *cloud-init*.
4. **Aplicação de configuração de rede:** a configuração entregue pelo *endpoint* `/network-config` foi servida como YAML válido e capturada para inspeção do estado das interfaces via `ip addr`.
5. **Atualização após mudança de estado:** após a reinicialização de uma instância, os metadados persistidos refletiram o novo estado dentro da janela de sincronização periódica.

Tabela 2. Resultados dos testes funcionais.

Propriedade	Resultado
Isolamento de metadados	Aprovado
Isolamento sob falsificação de IP (X-Forwarded-For)	Aprovado
Execução de <i>user-data</i>	Aprovado
Entrega de <i>network-config</i>	Aprovado
Propagação de estado após reinício	Aprovado

Todas as propriedades foram atendidas. Em particular, uma inicialização real de *cloud-init* selecionou o *datasource* NoCloud de rede, obteve seus metadados e *user-data* do serviço (respostas HTTP 200) e concluiu com `cloud-init status: done`, evidenciando a compatibilidade funcional de ponta a ponta.

4.3. Avaliação de Desempenho

A avaliação de desempenho concentrou-se em três métricas: latência de resposta da API, latência de sincronização e escalabilidade sob variação do número de instâncias.

4.3.1. Latência de Resposta da API

A latência foi medida para os quatro *endpoints* do serviço. A Tabela 3 apresenta os percentis p50, p95 e p99 obtidos com 5000 requisições por *endpoint*. O *endpoint* `/vendor-data` respondeu com código HTTP 404 por não haver dado de *vendor* configurado no cenário (comportamento correto, tolerado pelo *cloud-init*), mas suas latências permanecem representativas do custo de processamento da requisição.

Tabela 3. Latência de resposta por *endpoint* (5000 requisições, concorrência 50).

Endpoint	p50 (ms)	p95 (ms)	p99 (ms)
<code>/meta-data</code>	13,2	53,7	83,2
<code>/user-data</code>	18,9	53,6	72,4
<code>/network-config</code>	11,4	47,2	72,4
<code>/vendor-data</code>	17,0	48,4	68,9

As latências medianas situam-se em torno de 11–19 ms e o percentil p99 abaixo de 90 ms para todos os *endpoints*, valores compatíveis com as janelas de tempo disponíveis durante a inicialização de instâncias.

4.3.2. Latência de Sincronização

A latência de sincronização (intervalo entre a criação de uma instância no Incus e a disponibilização de seu registro de metadados) foi medida em aproximadamente **20,5 s**. Esse valor é consistente com a janela do ciclo de sincronização periódica adotado (*polling* a cada 10 s) somada ao tempo de inicialização de rede da instância, correspondendo tipicamente a cerca de dois ciclos de sincronização.

4.3.3. Escalabilidade

Para avaliar o comportamento com o aumento do número de instâncias simultâneas, mediu-se a latência de resposta do *endpoint* /meta-data para configurações de 5 a 200 contêineres. A hipótese inicial era de que, por utilizar SQLite como mecanismo de persistência, o serviço apresentasse degradação de desempenho sob alta concorrência de escrita, dado que o SQLite serializa operações de escrita OpenStack Foundation [2024]. A Tabela 4 apresenta os resultados.

Tabela 4. Escalabilidade: latência de /meta-data e memória por número de instâncias.

Instâncias	p50 (ms)	p95 (ms)	p99 (ms)	Erros	Mem. (MB)
5	16,2	60,3	92,4	0,0%	1152
10	13,1	52,7	78,2	0,0%	1325
25	12,4	51,8	77,2	0,0%	1373
50	12,9	52,0	80,8	0,0%	1733
100	14,4	58,6	92,1	0,0%	1763
150	12,8	53,4	82,2	0,0%	1638
200	12,8	54,2	87,2	0,0%	1711

Os dados **não corroboram** a hipótese de degradação por serialização de escrita nessa faixa de escala: a latência permaneceu estável (p50 entre 12 e 16 ms; p99 entre 77 e 98 ms) e a taxa de erro foi nula em todas as configurações, até 200 instâncias sobre 4 vCPUs. Esse resultado decorre da natureza do fluxo predominante, majoritariamente requisições de leitura (GET) de metadados, enquanto as escritas de sincronização ocorrem apenas a cada 10 s e são absorvidas pelo modo WAL (*Write-Ahead Logging*) e pelo *timeout* de ocupação configurados no SQLite. Conclui-se que a serialização de escrita não é um gargalo no perfil de carga avaliado; uma eventual degradação exigiria contagens de instâncias ou taxas de escrita substancialmente superiores, o que se registra como limite medido e não como tendência de degradação linear.

4.4. Avaliação de Alta Disponibilidade

Para avaliar o mecanismo de alta disponibilidade baseado em consenso RAFT, provisionou-se um *cluster* de três nós (e2-standard-2) com endereços internos fixos,

um dos quais designado como nó de inicialização (*bootstrap*). Adotou-se a topologia de **réplicas sobre um Incus compartilhado**: os três serviços de metadados apontam para a mesma instância do Incus, de modo que o líder sincroniza o estado a partir de uma visão única de instâncias e o replica aos seguidores via *log* do RAFT. A Tabela 5 resume os resultados.

Tabela 5. Resultados da avaliação de alta disponibilidade (*cluster* de 3 nós).

Propriedade	Resultado
Formação do <i>cluster</i>	1 líder + 2 seguidores
Replicação de estado	instância criada no líder presente nos 3 nós
Reeleição após falha do líder	2,40 s
Disponibilidade dos dados após <i>failover</i>	preservada (sem remoção indevida)
Reintegração de nó	nó reiniciado reingressa como seguidor

O *cluster* formou-se corretamente, com eleição de um único líder. Uma instância criada no Incus compartilhado teve seu registro replicado aos três nós. Ao encerrar-se abruptamente o serviço do líder (mantendo-se o Incus compartilhado ativo), um novo líder foi eleito em **2,40 s**, valor coerente com os *timeouts* configurados (*heartbeat* de 1 s e eleição de 1 s), e os dados replicados permaneceram disponíveis nos nós sobreviventes, sem remoção indevida. Um nó previamente encerrado foi capaz de reingressar ao *cluster* e recuperar o estado a partir do *log*.

Observou-se que a correteza da rotina de reconciliação (que remove registros de instâncias ausentes do Incus) depende da topologia adotada: sob a topologia de Incus compartilhado, todo nó que possa assumir a liderança observa o mesmo conjunto de instâncias, de modo que a reconciliação é correta e o *failover* não provoca perda de dados. Uma topologia alternativa, com um Incus por nó (adequada a cenários de borda distribuída), exigiria reconciliação restrita às instâncias de cada nó de origem, de forma que um nó jamais removesse instâncias pertencentes a outro.

4.5. Comparação com Provedores de Nuvem

Para contextualizar as capacidades do serviço proposto, a Tabela 6 compara os campos de metadados disponibilizados com os oferecidos pelo AWS Instance Metadata Service Amazon Web Services [2024a], pelo serviço de metadados do Google Cloud Platform Google Cloud [2024] e pelo serviço de metadados do OpenStack OpenStack Foundation [2024].

O serviço proposto cobre os campos essenciais para a operação dos módulos mais comuns do *cloud-init* (*set_hostname*, *users-groups*, *write-files*, *runcmd* e configuração de rede), oferecendo também entrega de *vendor-data* por instância com *fallback* global. Campos específicos de provedores públicos, como tipo de instância e atributos de faturamento, não são expostos, por não se aplicarem ao contexto de *homelab* e computação de borda visado.

4.6. Discussão

Os resultados obtidos permitem avaliar em que medida o serviço de metadados proposto atende aos requisitos de um ambiente Incus de pequena escala. Do ponto de vista funci-

Tabela 6. Comparação de campos de metadados (✓ suportado; ✗ não suportado; P: parcial).

Campo	Proposto	AWS	GCP	OpenStack
instance-id	✓	✓	✓	✓
local-hostname	✓	✓	✓	✓
local-ipv4	✓	✓	✓	✓
Chaves SSH públicas	✓	✓	✓	✓
availability-zone	✓	✓	✓	P
Tipo de instância	✗	✓	✓	✓
user-data	✓	✓	✓	✓
vendor-data	✓	✗	✗	✓
network-config	✓	P	P	✓

onal, o serviço demonstrou corretude no isolamento de metadados (inclusive sob tentativa de falsificação de origem) e na entrega de configurações efetivamente aplicadas pelo *cloud-init*, validando a premissa de compatibilidade com o *datasource* de rede Canonical Ltd. [2024a]. Do ponto de vista de desempenho, a natureza leve da implementação traduziu-se em latências de dezenas de milissegundos e em escalabilidade estável até 200 instâncias. Do ponto de vista de disponibilidade, o mecanismo de consenso demonstrou eleição e recuperação de líder da ordem de segundos.

Entre as limitações do serviço proposto, destacam-se:

- **Escalabilidade do SQLite:** embora não se tenha observado degradação até 200 instâncias, o SQLite serializa operações de escrita, o que poderia tornar-se um gargalo sob taxas de criação e destruição de instâncias muito superiores às avaliadas. Essa característica é inerente à escolha por uma solução sem dependências externas e é aceitável no perfil de carga de *homelab* e computação de borda de pequena escala.
- **Cobertura parcial de campos de metadados:** o serviço implementa o subconjunto de campos necessários aos módulos mais comuns do *cloud-init*, mas não cobre todos os campos de provedores como AWS e GCP Amazon Web Services [2024a]; Google Cloud [2024]. Cargas que dependam de campos específicos podem requerer extensões.
- **Dependência da topologia na alta disponibilidade:** as garantias de disponibilidade sem perda de dados no *failover* pressupõem a topologia de réplicas sobre um Incus compartilhado; cenários com um Incus por nó demandam reconciliação por nó de origem.

Quanto às ameaças à validade, os experimentos de desempenho e escalabilidade foram executados em host único, o que não captura efeitos de rede real entre *hypervisor* e instâncias em topologias amplamente distribuídas; a avaliação de alta disponibilidade, contudo, já emprega um *cluster* de três nós. As instâncias utilizadas são contêineres do tipo sistema (*system containers*), que compartilham o *kernel* do host, podendo apresentar latências de inicialização distintas das de máquinas virtuais completas. Por fim, a variabilidade do intervalo de sincronização periódica introduz uma janela de inconsistência cujos efeitos dependem da frequência de mudanças de estado no ambiente em produção.

Em conjunto, os achados sustentam o objetivo definido na Seção 1: prover, para ambientes Incus, um serviço de metadados compatível com *cloud-init*, leve e com suporte a alta disponibilidade.

5. Conclusão

Este trabalho apresentou o projeto e a implementação de um serviço de metadados compatível com *cloud-init* para o Incus. O serviço preenche uma lacuna no ecossistema de gerenciamento de contêineres e máquinas virtuais, oferecendo um endpoint de metadados independente que as instâncias podem consultar para obter suas configurações de inicialização, de forma análoga ao que provedores de nuvem pública disponibilizam nativamente. Foram implementados os endpoints REST exigidos pelas fontes de dados do *cloud-init* (*meta-data*, *user-data*, *vendor-data* e *network-config*), além de um sistema de sincronização orientado a eventos que mantém os dados das instâncias atualizados no banco de dados. A identificação das instâncias por endereço IP permite uma operação transparente, sem necessidade de agentes ou configurações adicionais dentro das instâncias. O serviço implementa ainda suporte a consenso distribuído via RAFT, habilitando implantações de alta disponibilidade com múltiplos nós coordenados por eleição de líder, replicação de log e aplicação determinística de comandos de escrita por meio de uma máquina de estados finita (FSM).

A avaliação experimental verificou a correteza funcional e o desempenho do serviço em cenários com diferentes números de instâncias simultâneas (de 5 a 200 contêineres), bem como o mecanismo de alta disponibilidade baseado em consenso RAFT em um *cluster* de três nós. Os resultados validaram a viabilidade da abordagem em ambientes Incus reais: as inicializações de *cloud-init* completaram-se com sucesso, as latências de resposta situaram-se na faixa de dezenas de milissegundos e a reeleição de líder após falha ocorreu em cerca de dois segundos. Algumas limitações foram identificadas. O uso do SQLite impõe restrições à concorrência de escrita em cenários de alta carga, ainda que não se tenha observado degradação até 200 instâncias. O conjunto de campos de metadados suportados também representa um subconjunto do que plataformas como AWS e GCP oferecem.

Como trabalhos futuros, destacam-se as seguintes direções: (i) adicionar suporte à convenção 169.254.169.254 Cheshire et al. [2005] por meio de *namespace* de rede ou *proxy* reverso, eliminando a necessidade de configuração explícita nas instâncias; (ii) ampliar o suporte a módulos e campos do *cloud-init* para maior compatibilidade com cargas de trabalho existentes; (iii) implementar autenticação baseada em sessões inspirada no IMDSv2 Amazon Web Services [2024b] para mitigar riscos de SSRF MITRE Corporation [2021]; (iv) substituir o mecanismo de sincronização por *polling* pela API de eventos do Incus, obtendo sincronização em tempo real com menor sobrecarga; e (v) realizar *benchmarks* de desempenho em cenários de maior escala. Estas melhorias consolidariam o serviço como uma solução robusta para ambientes privados que demandam automação de infraestrutura baseada em *cloud-init*.

Referências

Amazon Web Services (2019). Add defense in depth against open firewalls, reverse proxies, and SSRF vulnerabilities with enhancements to the EC2 instance metadata service.

- Amazon Web Services (2024a). Instance metadata and user data — Amazon EC2 documentation.
- Amazon Web Services (2024b). Use IMDSv2 — Amazon EC2 documentation.
- Artač, M., Borovšak, T., Di Nitto, E., Guerriero, M., and Tamburri, D. A. (2017). Devops: Introducing infrastructure-as-code. *IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, pages 497–498.
- Bollig, E. F., Allan, G. T., Lynch, B. J., Huerta, Y. A., Mix, M., Munsell, E. A., Benson, R. M., and Swartz, B. (2018). Leveraging OpenStack and Ceph for a controlled-access data cloud. In *Proceedings of the Practice and Experience on Advanced Research Computing (PEARC '18)*, Pittsburgh, PA, USA. ACM.
- Bresnahan, J., Freeman, T., LaBissoniere, D., and Keahey, K. (2011). Managing appliance launches in infrastructure clouds. In *Proceedings of the 2011 TeraGrid Conference: Extreme Digital Discovery*, Salt Lake City, UT, USA. ACM.
- Canonical Ltd. (2024a). cloud-init documentation.
- Canonical Ltd. (2024b). EC2 datasource — cloud-init documentation.
- Canonical Ltd. (2024c). LXD datasource — cloud-init documentation.
- Canonical Ltd. (2024d). LXD documentation.
- Canonical Ltd. (2024e). Networking config version 2 — cloud-init documentation.
- Canonical Ltd. (2024f). Nocloud — cloud-init documentation.
- Cheshire, S., Aboba, B., and Guttman, E. (2005). Dynamic configuration of IPv4 link-local addresses. RFC 3927, IETF.
- Cloud Native Computing Foundation (2024a). OpenTelemetry — an observability framework for cloud-native software.
- Cloud Native Computing Foundation (2024b). Prometheus — monitoring system & time series database.
- Gin-Gonic Contributors (2024). Gin web framework.
- Google Cloud (2024). About VM metadata — Google Cloud documentation.
- Gräber, S. (2023). Incus as an alternative to LXD.
- Guerriero, M., Garriga, M., Tamburri, D. A., Palomba, F., and Lago, P. (2019). Adoption, support, and challenges of infrastructure-as-code: Insights from industry. *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 580–589.
- HashiCorp (2024). hashicorp/raft — golang implementation of the raft consensus protocol.
- Hipp, D. R. (2024). SQLite — small. fast. reliable. choose any three.
- Linux Containers (2024). Incus — system container and virtual machine manager.
- Microsoft (2024). Azure instance metadata service.
- MITRE Corporation (2021). T1552.005: Unsecured credentials — cloud instance metadata API.
- Novaes Neto, G., Madnick, S., Moraes, A., and Borges, N. (2020). A case study of the capital one data breach. *MIT Sloan School Working Paper*, (2020-07).
- Ongaro, D. and Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319.
- OpenStack Foundation (2024). Metadata service — OpenStack documentation.
- Sousa, J. V. d. (2018). *Computação em Nuvem no Contexto das Smart Grids: Uma Aplicação para Auxílio à Localização de Falhas em Sistemas de Distribuição*. Tese

de doutorado, Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos.

Teppan, H., Flå, L. H., and Jaatun, M. G. (2022). A survey on infrastructure-as-code solutions for cloud development. In *2022 IEEE 13th International Conference on Cloud Computing Technology and Science (CloudCom)*, Bangkok, Thailand. IEEE.